

UNIVERSITY OF **Baguio**

SCHOOL OF INFORMATION TECHNOLOGY



WEBSYS1

Web Systems and Technologies 1

Topic Overview

This module introduces students to Object-Oriented Programming (OOP) in PHP, a programming paradigm that allows developers to model real-world entities as objects. Students will begin by understanding functions as reusable blocks of code, then progress to classes and objects, learning how to initialize objects using constructors and manage object lifecycle using destructors.

The module also covers methods, which are functions inside classes that manipulate object data. Practical exercises will reinforce concepts through string formatting, array operations, password generation, age calculation, and existence checks, helping students connect OOP concepts to real-life problem-solving in PHP.

Learning Outcomes

At the end of the lesson, the student should be able to:

1. Familiarize with PHP functions and understand their purpose in code reusability.
2. Create and use custom PHP functions to perform specific tasks.
3. Define PHP classes with properties and methods to model real-world entities.
4. Instantiate objects from PHP classes and access their properties and methods.
5. Implement constructors to initialize object properties during object creation.
6. Implement destructors to manage object cleanup at the end of a script.
7. Define and call methods within classes to perform actions using object properties.
8. Apply PHP OOP concepts in practical exercises such as string formatting, array operations, password generation, age calculation, and item existence checking.

I. Functions

1. A function is a reusable block of code that performs a specific task.
2. Functions help avoid code repetition, make programs easier to read, and can take inputs (parameters) and return outputs.

PHP Syntax for function

```
function functionName($param1, $param2) {  
    // Code to execute  
    return $result;  
}
```

Explanation of Syntax:

- function - Keyword to define a function
- functionName - Name of your function (must be unique in scope)
- (\$param1, \$param2) - Input parameters (optional)
- return \$result; - Sends the output back to the code that called the function

Example 1: Greeting Function

```
function greet($name) {  
    return "Hello, $name!";  
}
```

```
}
echo greet("John"); // Output: Hello, John!
```

Explanation:

1. greet(\$name) - Function called greet accepts a single parameter \$name.
2. return "Hello, \$name!" - Returns a string with the name included.
3. echo greet("John") - Calls the function and prints: Hello, John!

PHP Built-in Functions

Function	Description
trim()	Removes whitespace from the beginning and end of a string.
strtoupper()	Converts a string to uppercase.
count()	Returns the number of elements in an array.
str_shuffle()	Randomly shuffles the characters in a string.
substr()	Returns a part of a string, starting at a specific index.
date()	Returns the current date or parts of it.
in_array()	Checks if a value exists in an array, returns true/false.

Exercise 1: String Formatter

Task: Format a name by trimming extra spaces and converting it to uppercase.

```
function formatName($name) {
    $trimmedName = trim($name);
    $upperName = strtoupper($trimmedName);

    return $upperName;
}
echo formatName(" john doe ");
```

Explanation Step-by-Step:

1. \$trimmedName = trim(\$name); - Removes spaces before and after the name.
2. \$upperName = strtoupper(\$trimmedName); - Converts the cleaned name to uppercase.
3. return \$upperName; - Sends the result back.
4. echo prints the final result.

Exercise 2: Count Items

```
function countItems($items) {
    return count($items);
}
echo countItems(["apple", "banana", "orange"]);
```

Explanation:

- count(\$items) - Returns how many elements are in the array.
- echo prints the number 3.

Exercise 3: Random Password Generator

```
function generatePassword($length) {  
    $chars = 'abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789';  
    $shuffled = str_shuffle($chars);  
    return substr($shuffled, 0, $length);  
}  
  
echo generatePassword(8);
```

Explanation:

1. \$chars - Contains letters and numbers for password.
2. str_shuffle(\$chars) - Randomizes characters.
3. substr(..., 0, \$length) - Extracts the first \$length characters to meet the required password length.

Exercise 4: Basic Age Calculator

```
function calculateAge($birthYear) {  
    return date("Y") - $birthYear;  
}  
  
echo calculateAge(2000);
```

Explanation:

- date("Y") - Returns current year as 4-digit number.
- date("Y") - \$birthYear - Calculates the age.

Exercise 5: Check if Item Exists

```
function isFruitAvailable($fruits, $fruitName) {  
    if (in_array($fruitName, $fruits)) {  
        return "Available";  
    } else {  
        return "Not Available";  
    }  
}  
  
echo isFruitAvailable(["apple", "banana", "grape"], "banana");
```

Explanation:

1. in_array(\$fruitName, \$fruits) - Returns true if fruit exists.
2. if-else - Returns "Available" or "Not Available" accordingly.

II: Classes

A class is a blueprint for objects. It defines properties (data) and methods (actions).

Syntax:

```
class ClassName {  
    public $property1;  
    public function method1() {  
        // Code here  
    }  
}
```

Example: Car Class

```
class Car {  
    public $brand = "Toyota";  
  
    public function displayBrand() {  
        echo $this->brand;  
    }  
}  
  
$car1 = new Car();  
$car1->displayBrand();
```

Explanation Step-by-Step:

1. public \$brand = "Toyota"; → Declares a public property with default value "Toyota".
2. displayBrand() → Method prints the property \$brand.
3. \$this->brand → \$this refers to the current object.
4. \$car1 = new Car(); → Creates an instance of the Car class.
5. \$car1->displayBrand(); → Calls method for this object.

III: Objects

Objects are instances of classes. They contain data (properties) and behavior (methods) defined in the class.

Already shown above with \$car1.

Example: Car Class

```
class Car {  
    public $brand = "Toyota";  
  
    public function displayBrand() {  
        echo $this->brand;  
    }  
}  
  
$car1 = new Car();  
$car1->displayBrand();
```

IV: Constructor

Special method called automatically when an object is created. Used to initialize properties.

Syntax:

```
class ClassName {  
    public function __construct($param) {  
        // Initialization code  
    }  
}
```

Example:

```
class Person {  
    public $name;  
  
    public function __construct($name) {  
        $this->name = $name;  
    }  
}
```

```
}
}

$person1 = new Person("Alice");
echo $person1->name;
```

Explanation:

- `__construct($name)` → Runs automatically during object creation.
- `$this->name = $name;` → Sets object's name property.

V: Destructor

Special method called automatically when an object is destroyed or script ends. Used for cleanup tasks like closing files or freeing memory.

Example:

```
class Logger {
    public function __destruct() {
        echo "Logger is being closed.";
    }
}

$log = new Logger();
```

Explanation:

- Destructor `__destruct()` runs at the end of script execution.

VI: Methods

Functions defined inside a class. They act on properties of the object.

Example: Calculator

```
class Calculator {
    public function add($a, $b) {
        return $a + $b;
    }
}

$calc = new Calculator();
echo $calc->add(5, 10);
```

Explanation:

- Method `add($a, $b)` takes two parameters and returns their sum.
- `$calc->add(5, 10)` calls this method for the object.

Summary

Concept	What It Does
Function	Reusable code block performing a task
Class	Blueprint for objects
Object	Instance of a class

Concept	What It Does
Constructor	Initializes object properties automatically
Destructor	Cleans up when object is destroyed
Method	Function inside a class